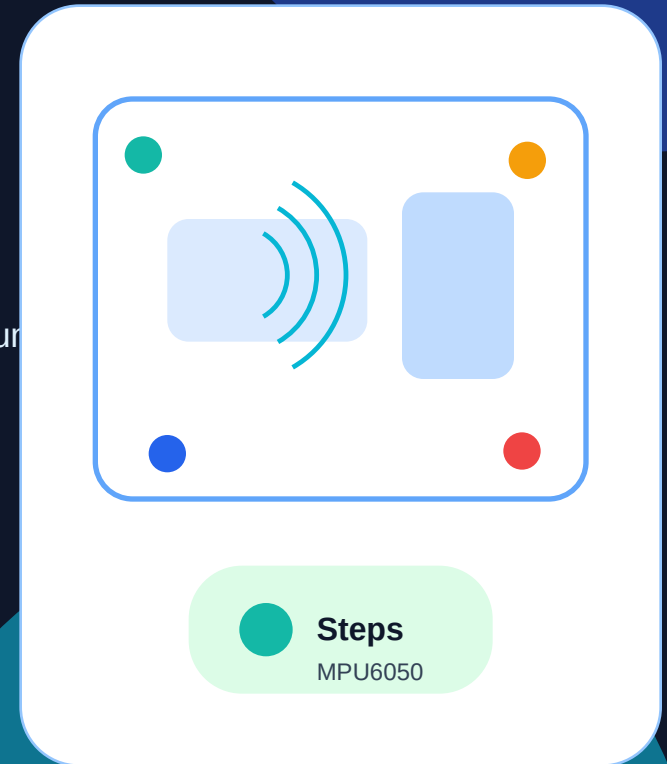


RoomSense

Smart Room & Activity Monitoring

Sensoren messen Raumklima, Bewegung und Aktivität. Die Daten werden gespeichert, analysiert und in einer modernen App visuell ausgewertet.



Kurzfassung: ESP32 + Sensoren -> API -> Datenbank -> Dashboard mit Scores, Trends und Alerts

Projektkern

Was RoomSense tut und warum es mehr ist als ein normales Sensor-Dashboard.

Ausgangslage

Problem

Innenräume werden oft schlecht gelüftet, zu dunkel, zu laut oder lange ohne Bewegung genutzt. Rohdaten allein helfen wenig - sie brauchen Kontext.

Systemidee

Lösung

RoomSense misst physikalische Daten, speichert sie strukturiert und berechnet daraus Scores, Empfehlungen und Ereignisse.

Demo-Effekt

Mehrwert

Die App zeigt nicht nur Werte, sondern bewertet Raumkomfort, Aktivität und potenzielle Störfaktoren visuell und verständlich.

Projektstatement

RoomSense analysiert, wie gesund, produktiv und aktiv ein Raum genutzt wird. Dafür werden Raumklima-, Umgebungs- und Bewegungsdaten erfasst, gespeichert und in einer Windows-fähigen Webapp ausgewertet.

physikalische Messdaten

Datenbank

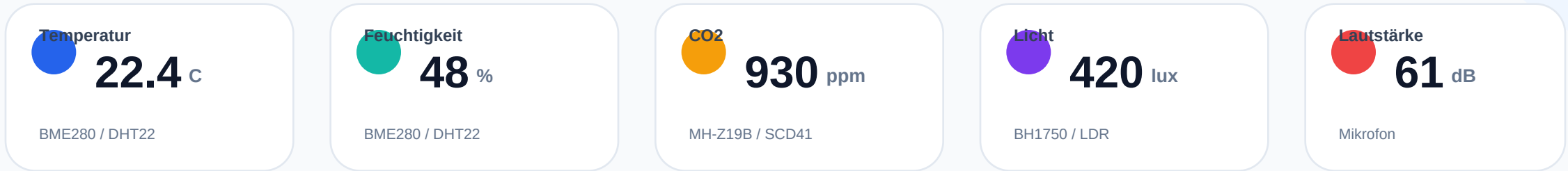
JavaScript-first

Windows-PC

Dashboard

Messkonzept

Raumdaten plus Aktivitätsdaten: dadurch entsteht ein interessanteres Gesamtbild.



Room Node

ESP32 liest Raumklima und Umgebung: Temperatur, Feuchte, CO2, Licht, Lärm, Bewegung. Messwerte werden zyklisch an die API gesendet.

kombiniert

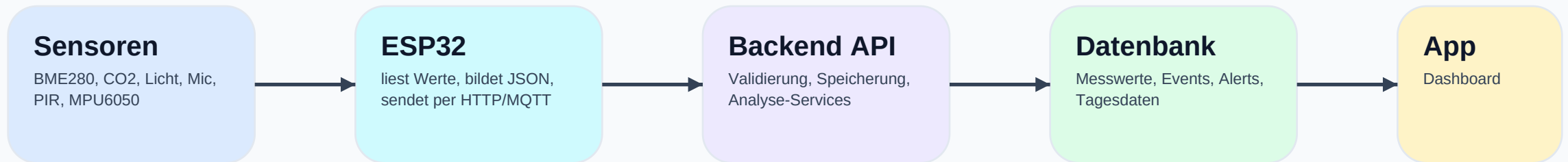
Activity Node

ESP32 + MPU6050 als kleiner Schritt-/Bewegungstracker. Erkennt Peaks in der Beschleunigung und erzeugt Schritt- und Aktivitätsdaten.

Wichtig: Schritte sollten über einen Beschleunigungssensor gemessen werden. Ein PIR-Sensor erkennt nur Bewegung im Raum, aber keine echten Schritte.

Systemarchitektur

Datenfluss von der physischen Messung bis zur visuellen Auswertung.



Beispielpayload vom ESP32

```
{
  "device": "RoomNode-01",
  "type": "CO2",
  "value": 930,
  "unit": "ppm",
  "timestamp": "2026-06-26T10:15:00"
}
```

Kommunikation

MVP: HTTP POST direkt an die API. Erweiterung: MQTT-Broker, wenn mehrere Sensorgeräte parallel publizieren sollen.

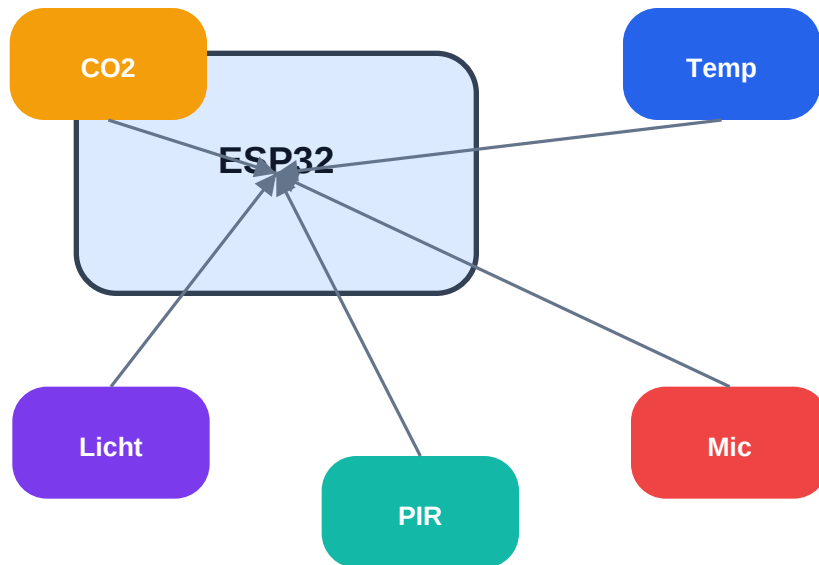
HTTP = einfach

MQTT = skalierbarer

Hardware-Skizze

Zwei kleine Messknoten reichen für eine starke Demo.

Room Node



Misst Raumklima und Umgebung. Sendet alle 5-30 Sekunden eine kleine JSON-Nachricht.

Activity Node



Schritte werden über Beschleunigungs-Peaks erkannt. Ergebnis: Schritte pro Minute, Aktivitätsstatus und Inaktivitätsphasen.

Datenbank-Alternativen

Nicht auf MS SQL festlegen: die Architektur bleibt austauschbar.

Option	Stärken	Schwächen	Empfehlung
MS SQL Server	Sehr gut für Windows/.NET, solide relationale Daten	Setup schwerer als SQLite/Postgres	Gut, wenn Microsoft-Umfeld gefordert ist
PostgreSQL	Kostenlos, stark, SQL, sehr flexibel	Etwas Admin-Aufwand	Beste offene Allround-Alternative
SQLite	Keine Serverinstallation, perfekt für lokale Demo	Nicht ideal für mehrere Clients/hohe Datenrate	Gut für Prototyp und Offline-App
InfluxDB	Optimiert für Zeitreihen, Sensorstreams, Aggregation	Anderes Datenmodell als klassisches SQL	Stark, wenn Messdaten im Fokus stehen
MongoDB	Flexible JSON-Dokumente, schnell für Prototypen	Weniger sauber für relationale Analysen	Okay, aber nicht erste Wahl für Scores
MariaDB/MySQL	Bekannt, breit verfügbar, SQL	Weniger spezialisiert für Zeitreihen	Solide, wenn schon vorhanden

Klare Empfehlung: Für maximale Projektwirkung PostgreSQL oder MS SQL. Für eine reine lokale Demo SQLite. Für technisch ambitionierte Sensor-Zeitreihen InfluxDB.

Programmiersprachen & App-Varianten

JavaScript zuerst, aber mit belastbaren Alternativen.

Empfohlener Stack: JavaScript-first

Frontend: React/Vite + Chart.js. Backend: Node.js + Express/Fastify. Desktop-Option: Electron oder Tauri. Datenbank: PostgreSQL, MS SQL, SQLite oder InfluxDB.

einheitliche Sprache

sehr gute Visualisierung

Windows-fähig

Variante	Technik	Bewertung
Webapp	React + Node.js API + DB	Beste Balance. Läuft im Browser auf Windows-PC.
Desktop-App	Electron + React + Node.js	Sehr anschaulich, echtes App-Gefühl, etwas größeres Paket.
Leichtgewicht-Desktop	Tauri + React	Schlanker, moderner, aber Rust-Tooling kommt dazu.
C#-Alternative	ASP.NET Core + Blazor + SQL Server	Sehr sauber im Microsoft-Umfeld.
Java-Alternative	Spring Boot + React/JavaFX	Möglich, aber für euer Ziel mehr Overhead.
Python-Alternative	FastAPI + Dashboard	Schneller Prototyp, weniger ideal für Windows-App-Charakter.

Datenmodell

Ein sauberes Schema macht spätere Auswertungen deutlich leichter.



Prinzip: Rohdaten bleiben unverändert gespeichert. Auswertungen wie Tageswerte, Scores und Alerts können separat berechnet und abgelegt werden.

Auswertung

Hier entsteht der eigentliche Projektwert: aus Messwerten werden Entscheidungen.

Score

Comfort Score

Bewertung von Temperatur, Feuchtigkeit, CO2, Licht und Lautstärke. Ergebnis: 0-100 Punkte für Raumkomfort.

Score

Activity Score

Schritte pro Zeitraum, Bewegungsphasen und Inaktivität. Ergebnis: Aktivitätslevel und Bewegungsempfehlung.

Alerts

Event Detection

System erkennt Grenzwertverletzungen: CO2 zu hoch, zu laut, lange inaktiv, Raum vermutlich belegt.

Beispiel-Regeln

- CO2 > 1000 ppm -> Lüften empfohlen
- CO2 > 1500 ppm -> kritische Luftqualität
- Lärmspitze > Durchschnitt + 25 dB -> Noise Event
- 45 Minuten keine Schritte -> Inaktivitätswarnung
- Feuchte > 65 Prozent und Temperatur niedrig -> Schimmelrisiko-Indikator

Score-Idee

Comfort = 30% CO2 + 25% Temperatur + 20%
Feuchte + 15% Lärm + 10% Licht

UI-Mockup

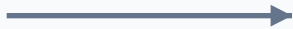
So kann die Windows-fähige Webapp visuell aussehen.



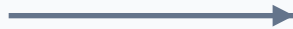
Umsetzungsplan

Realistisch schneiden: erst MVP, dann Erweiterungen.

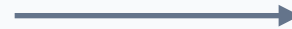
1



2



3



4

Sensor-MVP

ESP32 liest Temperatur, Feuchte, Licht und CO2.

API + DB

POST-Endpunkte, DB-Schema, Speicherung.

Dashboard

Live-Karten, Charts, Filter, Tageswerte.

Activity

MPU6050-Schritte, Aktivitätsanalyse, Alerts.

MVP-Scope

- 1 Room Node
- 1 Activity Node
- Datenbank austauschbar
- Dashboard mit Livewerten
- 3 Charts
- Comfort Score + Activity Score
- Alerts und Tageszusammenfassung

Nice-to-have

- MQTT statt HTTP
- Export als CSV/PDF
- Mehrere Räume
- Heatmap für Aktivität
- Admin-Seite für Grenzwerte
- Dark Mode
- Automatische Tagesberichte

Entscheidungsvorschlag

So würde ich das Projekt konkret pitchen.

Finaler Projektzuschnitt

RoomSense ist ein Smart-Monitoring-System, das physikalische Raumdaten und Aktivitätsdaten erfasst, flexibel in einer Datenbank speichert und über eine JavaScript-basierte Windows-fähige Webapp visuell auswertet.

Bereich	Empfehlung
Sensorik	ESP32 + BME280/DHT22 + CO2-Sensor + BH1750/LDR + Mikrofon + MPU6050
Backend	Node.js + Express/Fastify API
Frontend	React + Chart.js
Desktop	zuerst Browser-Webapp, optional Electron oder Tauri für Windows-App
Datenbank	PostgreSQL oder MS SQL; SQLite für lokale Demo; InfluxDB als ambitionierte Zeitreihen-Option
Auswertung	Comfort Score, Activity Score, Lüften-Alarm, Noise Events, Inaktivitätswarnung

Technische Referenzen

Offizielle Quellen für Frameworks, Datenbanken und Visualisierung.

[1] Node.js: <https://nodejs.org/en/about>

[2] Electron: <https://electronjs.org/>

[3] Tauri: <https://v2.tauri.app/start/>

[4] Chart.js Docs: <https://www.chartjs.org/docs/>

[5] Microsoft SQL Server Docs: <https://learn.microsoft.com/en-us/sql/sql-server/>

[6] PostgreSQL Docs/About: <https://www.postgresql.org/docs/> und <https://www.postgresql.org/about/>

[7] InfluxDB Docs: <https://docs.influxdata.com/>

[8] ASP.NET Core Blazor Docs: <https://learn.microsoft.com/aspnet/core/blazor/>

Hinweis: Die PDF ist als Konzept- und Vorstellungsunterlage gedacht. Konkrete Sensorbibliotheken und Schwellwerte sollten im Prototyp kalibriert werden.